Practical 3a. AR(p) Models

1. Simulating AR(p) models:

Type in the following commands:

- n<-1000
- x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
- plot(x,type="l")

With this, we generated an AR (2) model.

Compute its auto-correlation function with the function acf:

- acf (x)

or

- k<-floor(log(n))
- acf (x, k)

Repeat the same exercise with different values of n, with a different number of parameters, and with different parameters of the AR(p).

2. Fitting AR(p) models:

Type in the following commands:

- n<-1000
- x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
- plot(x,type="l")
- s1<-arima(x, order=c(2,0,0), method="ML")
- s1

We obtain an estimation of the AR(2) parameters and the intercept. Assess the quality of the fit, and try with larger values of n.

Do it several times simulating new series with different parameters.

An alternative method is the following:

s2<- ar(x, order.max=2, method=”mle”)
s2

Try order. max=3.

3. Validation:

Try the following commands:

- n<-1000
- x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
- plot(x,type="l")
- s<-arima(x, order=c(2,0,0), method="ML")
- s

- y<-resid(s)
- plot(y,type=”l”)
- h<-floor(log(n))
- Box.test(x,lag=h, type=c("Ljung-Box"))
- Box.test(y,lag=h,type=c("Ljung-Box"))

Note that, unlike y, x is not an IID noise process. If the adjustment is correct, the series y ought to be an IID process. Plot the acf of y. An alternative method for the last three instructions is the following:

- h<-floor(log(n))
- tsdiag(s,gof.lag=h)

4. Selection of the best AR(p) model:

Please, type in:

- n<-1000
- x<-arima.sim(list(ar = c(0.8897, -0.4858), sd = sqrt(0.1796)), n)
- plot(x,type="l")
- ss1<-ar(x, order.max=3, method=”mle”)
- ss1

The instruction selects the AR model, AR(1), AR(2) or AR(3), that best fits the data x, using the Akaike method, which we will see this in next lectures. The solution should be an AR(2).

5. A complete analysis with real data:

Please, type in:

- x<-LakeHuron
- x

We have the dataset of the yearly level of Lake Huron from years 1875 to 1972.

Plot the series using:

- plot (x, ylab=”depth”, xlab=”times”)

Do

- lag.plot (x, lag=4, do.lines=F)

These are plots of (x(i-1), x(i)), then (x(i-2), x(i)), and so on. Notice that the linear trend disappears as we increase the lag.

Do

- acf (x)

To fit a model AR(1) to x we do

- s1<-ar(x, order.max=1, method=”yw”)
- s1

or

- s2<-ar(x, order.max=1, method=”mle”)
- s2

yw and mle denote different methods of estimation of the parameters. Yw stands for Yule-Walker and mle for Maximum Likelihood Estimation.

More generally, to fit an AR(p) model to x we do the following:

- s3<-ar(x, method="mle")
- s3

or

- s4<-ar(x, method="yw")
- s4

Compare the results of s3 or s4. Note that both propose an AR(2) with similar parameters.

Practice 3b: Simulating and fitting ARMA models

The goal of this practical is to fit an ARMA model to a given dataset, and to validate whether it is a good model for the specific dataset.

Outline:
1. Simulation of a model.
2. Heuristic identification of a model. Auto-correlation and partial auto-correlation functions.
3. Fitting a model.
4. Validation.

1. Model Simulation.

Do

- SIM1<-arima.sim(list(order=c(1,0,0), ar=0.7),n=1000)
- SIM2<-arima.sim(list(order=c(0,0,2), ma=c(1.5,0.75)),n=1000)
- SIM3<-arima.sim(list(order=c(2,0,1), ar=c(0.3,0.2), ma=0.3), n=1000)

We have simulated an AR(1) model with parameter 0.7, an MA(2) model with parameters –1.5 and –0.75, and an ARMA(2,1) model with parameters AR 0.3 and 0.2 and parameter MA –0.3. Plot the charts of SIM1, SIM2 and SIM3.

2. Heuristic identification of the model. Auto-correlation and partial auto-correlation functions.

Here we want to compute and represent graphically functions acf and pacf of our series. If x represents any of our series, we have to write

- acf(x,k)
- pacf(x,k)

where k represents the number of represented lags. Recall that k has to be at most one third of n, the number of simulated data. Some authors recommend at most k=ln(n).

Recall that an AR(p) model has an exponentially decreasing acf and a pacf that is null after p lags. An MA(q) model has an exponentially decreasing pacf and an acf that is null after q lags.

3. Fitting a model.

Observe the graphics of acf and pacf of the three series and deduce heuristically a good model. Afterwards fit the proposed model using the instruction

- S1<-arima(SIM3, order=c(1,0,0), method="ML")
- S1

And similarly, for the other two. Check if the estimations are good.

4. Validation.

We can apply the following instructions to S3:

- x<-SIM3
- h<-log(n)
- Box.test (x, lag = h, type = c("Ljung-Box"))

Here we reject the null hypothesis; SIM3 is not an IID noise!
But we can compute:

two main things of this practical:
1. Fitting
2. Forecasting

The role of AR & MA models is to model stationary processes —> expected value has to be constant and covariance has to be symmetric, and variance has to be also something that depends on the number of parameters we have (p)

- y<-resid (S3)
- Box.test (y, lag =h, type = c("Ljung-Box"))

How to check for stationarity —> ADF (dickey fulls test)

and now the p-value is greater than 0.05 and so we can accept the series of residuals is an IID noise. Thus, we validate our ARMA(2,0,1) model. A faster way to do it is using the instruction

- tsdiag(S3, gof.lag=20)

Check the plots. The third one should show whether the data has been well fitted. The p-values for all the lags are large. Option gof.lag indicates until what order we compute the statistics of Ljung-Box test.

5. A complete example:

AR(p) —> [Ey_j]=0 or mu, covariance=gamma(l). I should still transform the mean around 0
order p = p parameters —> one sample is evaluated based on weighted sum of previous p samples (so p weights, parameters) + the noise. If you fit the AR, it will give you the theta, the noise, etc. capture dependence between the samples

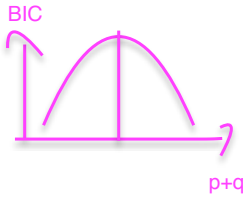
Do:

MA —> caputres moving average, how much does the series change —> capture dependence between the sample & noise. current sample depends on current noise and previous noise

- n<-10000
- h<-log(n)
- s3<-arima.sim(list(order=c(2,0,1),ar=c(0.3,0.2),ma=0.3), n)
- plot(s3, type="l")
- acf(s3)
- pacf(s3)
- p3<-arima(s3,order=c(2,0,1),method="ML")
- p3
- Box.test(p3,lag=h,type=c("Ljung-Box"))
- y<-resid(p3)
- Box.test(y,lag=h,type=c("Ljung-Box"))

So often we combine them.

How to know what is the right number of parameters p & q? AIC or BIC



6. Roots of a polynomial:

Given a polynomial 1+a_1 x+a_2 x^2+⋯⋯+a_n x^n we can obtain its roots using the R instruction:

- Polyroot (c(1,a_1,….,a_n))

Points 7 and 8 are not meant to be done in R but in python

7. Download the daily closing prices for Siemens Energy and store them to a SE.csv file.

- Load the CSV and compute **daily log returns**.
- Check if the returns are **stationary** (hint: use ADF test).
- Plot the **ACF and PACF** of the returns.
- Fit an **ARMA(p,q)** model to the returns (choose p and q based on ACF/PACF).
- Interpret the AR and MA coefficients. What do they say about short-term dependence in returns?
- Forecast the next **5 days of returns** and visualize them vs historical returns.
- Compare the performance of ARMA(1,1) vs ARMA(2,1) using **AIC/BIC**.

8. ARMA for Weather Data. Download a CSV file with the daily mean temperatures of your favourite city.

- Load the CSV and visualize the time series. Is there any visible trend or seasonality?
- Difference the series to make it **stationary** if needed.
- Plot **ACF and PACF** of the differenced series.
- Fit an **ARMA(p,q)** model to the stationary temperature changes.
- Forecast the next **7 days of temperature changes**.
- Convert the predicted changes back to **temperature predictions**.

- Plot **actual vs fitted values** for the last 30 days of your dataset.
- **Optional challenge:** Try using a **seasonal ARIMA (SARIMA)** if there is weekly or yearly seasonality in temperature.